

Lab 4 - Advanced CUDA

Advanced Computer Architecture

University of Minho

The Lab 4 focus on the development of efficient CUDA code by covering some of its advanced features that have a relevant impact on performance. Use a Deucalion node with NVidia GPUs (by specifying the partition `normal-gpu` when submitting a job). Compile the code in the frontend.

This lab tutorial includes exercises to be solved during the two next lab classes. Copy the source files, as usual, from `/projects/F202500010HPCVLABUMINHO/ACA/lab4` to your home directory in Deucalion:

```
cp -r /projects/F202500010HPCVLABUMINHO/ACA/lab4 .
```

To load the compiler in the environment use the following commands (do not forget to also use them in your submission script):

GNU Compiler: `module load GCC/12.3.0.`

CUDA Environment: `module load CUDA/12.4.0.`

You can use other GNU/CUDA combinations at your own risk. Use either `nvprof` or `nsys` command line tools to profile GPU code if necessary.

1 Asynchronous Computing

Goals: to develop skills in asynchronous memory copies and kernel launches.

Algorithm 1 Pseudocode for a 1D stencil.

```
for all  $E$  in  $Vector$  do
  for all  $Xi$  in radius  $R$  of  $E$  do
     $OutVector[E] += Xi$ ;
  end for
end for
```

Lab 1.1 Complement this skeleton with the GPU code (kernel) to compute a 1D stencil on a 2D matrix. Remember that the CUDA kernel is coded in such a way that it should be executed

by a single thread, using its *id*. The CUDA runtime will assign the kernel to a set of threads when it is called, so no explicit parallelism must be present in the code.

The stencil kernel should compute a single row of the 2D matrix each time. Complete the code to copy a one row at a time asynchronously, using a double buffer approach so that the stencil processes a run while another is copied. Run the GPU code and measure and record its execution time.

2 Multi-GPU Processing

Goals: to comprehend how to use multiple GPUs concurrently.

Lab 2.1 Consider the one dimensional stencil code from the previous exercise. Complete the provided code skeleton, ensuring that half of each matrix row is processed by one of two GPUs. Run the GPU code and measure and record its execution time.

Lab 2.2 Adapt the code of Lab 2.1 so that each GPU processes the whole row, i.e., each GPU processes a row simultaneously. Each GPU should have access to its double-buffer, as in Lab 1.1 and 2.1. The code must be prepared to use any number of GPUs, as configured by the user (either hardcoded or by passing an argument to the program).

Run the code using one to four GPUs (maximum available in a Deucalion's node), measure and record its execution time. How does this compare to previous results? Is there a matrix size for this code that benefits from multi-GPU usage?

3 Dynamic Asynchronous Parallelism

Goals: to comprehend how to use dynamic parallelism coupled with asynchronous computing.

Lab 3.1 Consider the one dimensional stencil code skeleton `stencil_dynamic.cu`. Complete the provided code skeleton, ensuring that a parent kernel is called for each row, which calls a child kernel to process the row.

Lab 3.2 Adapt the code of Lab 3.1 by completing the code skeleton `stencil_dynamic_async.cu`. The code should now asynchronously call a parent kernel for each row, in a independent stream (whenever possible), allowing multiple rows to be processed simultaneously.